LONDON METROPOLITAN UNIVERSITY

Islington college
(इस्लिङ्टन कलेज)

CS4001NI Programming

50% Individual Coursework

2025-26
Autumn
Credit: 30

Student Name: Aryush Aryal
London Met ID: 25029573
College ID: NP01AI4A250161

Assignment Due Date: April 17, 2026
Assignment Submission Date: April 17, 2026
Module Leader: Mr. Bikram Poudel
Word Count: 2187

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

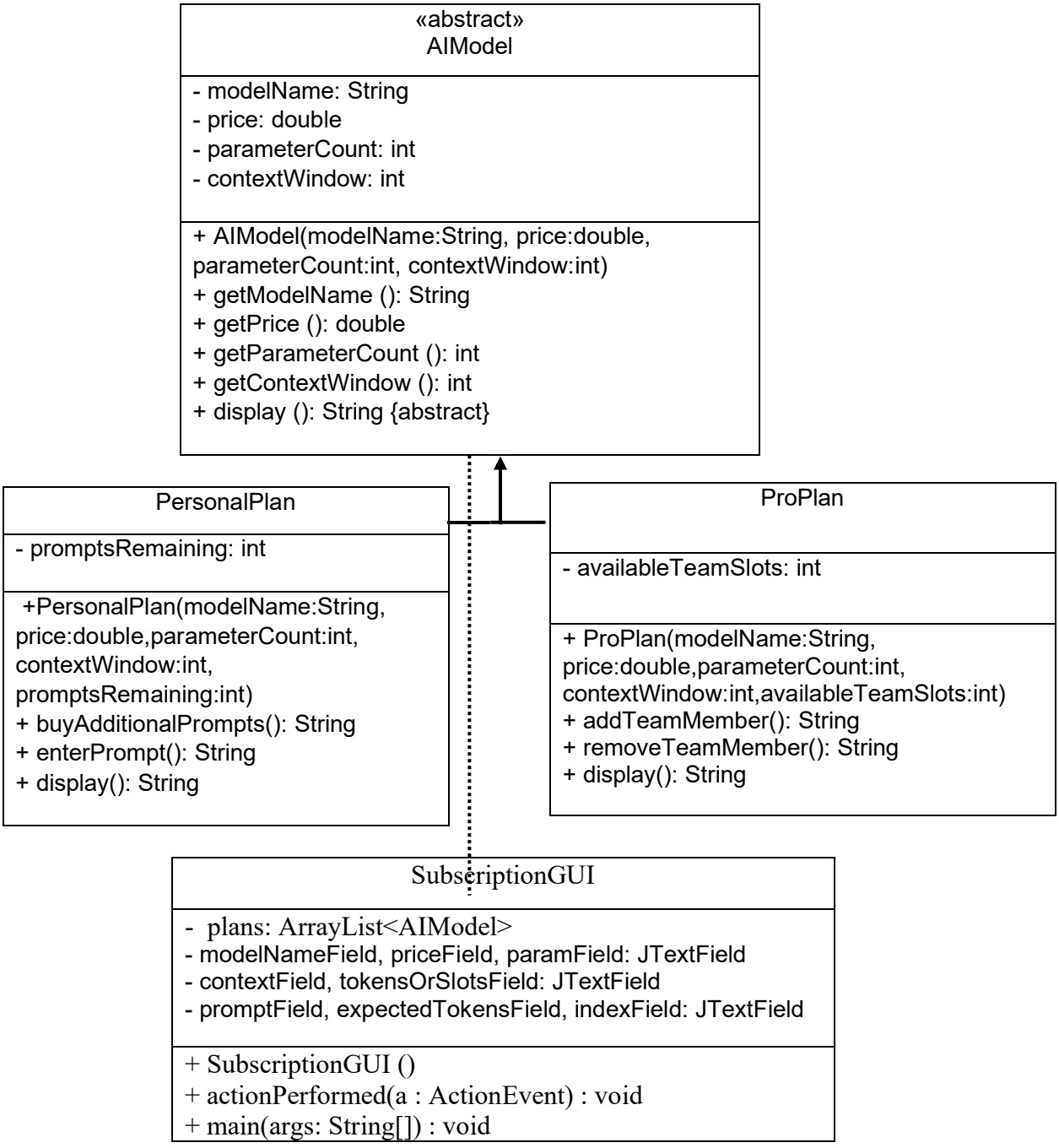
1. Introduction

This report outlines the design and implementation of an AI Subscription Management System created for the Programming module. The primary objective of this assignment is to use object-oriented programming techniques, create a GUI with Java, and perform file I/O operations. (Barnes & Kölling, 2021)

The system features an abstract parent class and child classes to represent different subscription plans, thus illustrating the use of abstraction, inheritance, and polymorphism. The user interface, based on Java Swing, enables users to make plans, submit prompts, add team members, and save or read data from a text file.

2. Class diagram

The AI Model subscription manager system is based on hierarchical inheritance. A visual class diagram showing the relationship and structure of these classes is given below (Barnes & Kölling, 2021). The class diagram follows standard UML notation to represent class structure and IS-A relationships between classes, supporting a clear understanding of the overall system design (Object Management Group, 2023).



3. Pseudocode

The following pseudocode describes the logic used in handling different button actions in the graphical user interface (Oracle, 2024).

3.1 Pseudocode for Class: AIModel

```
CLASS AIModel (Abstract)

VARIABLES
   modelName : String
   price : double
   parameterCount : int
   contextWindow : int

CONSTRUCTOR AIModel(modelName, price, parameterCount, contextWindow)
   this.modelName ← modelName
   this.price ← price
   this.parameterCount ← parameterCount
   this.contextWindow ← contextWindow
END CONSTRUCTOR

METHOD getModelName()
   RETURN modelName
END METHOD

METHOD getPrice()
   RETURN price
END METHOD

METHOD getParameterCount()
   RETURN parameterCount
END METHOD

METHOD getContextWindow()
   RETURN contextWindow
END METHOD

ABSTRACT METHOD display()
END METHOD

END CLASS
```

3.2 Pseudocode for Class: ProPlan

```
CLASS ProPlan EXTENDS AIModel

VARIABLES
   availableTeamSlots : int

CONSTRUCTOR ProPlan(modelName, price, parameterCount, contextWindow,
availableTeamSlots)
   Call AIModel constructor
   this.availableTeamSlots ← availableTeamSlots
END CONSTRUCTOR

METHOD addTeamMember(memberName)
   IF availableTeamSlots > 0 THEN
      availableTeamSlots ← availableTeamSlots − 1
      RETURN "Team member added successfully"
   ELSE
      RETURN "No available team slots"
   END IF
END METHOD

METHOD display()
   RETURN model information and availableTeamSlots
END METHOD

END CLASS
```

3.3 Pseudocode for Class: PersonalPlan

```
CLASS PersonalPlan EXTENDS AIModel

VARIABLES
   availableTokens : int

CONSTRUCTOR PersonalPlan(modelName, price, parameterCount, contextWindow,
availableTokens)
   Call AIModel constructor
   this.availableTokens ← availableTokens
END CONSTRUCTOR

METHOD enterPrompt(promptText, expectedTokens)
   Split promptText into words
   inputTokens ← number of words
   totalTokens ← inputTokens + expectedTokens

   IF totalTokens > contextWindow THEN
      RETURN "Token limit exceeds context window"
   ELSE IF availableTokens >= totalTokens THEN
      availableTokens ← availableTokens − totalTokens
      RETURN "Prompt submitted successfully"
   ELSE
      RETURN "Not enough available tokens"
   END IF
END METHOD

METHOD display()
   RETURN model information and availableTokens
END METHOD

END CLASS
```

3.4 Pseudocode for Class: SubscriptionGUI

```
CLASS SubscriptionGUI

VARIABLES
   plans : ArrayList of AIModel
   Text fields for user input
   Buttons for actions
   Output text area

CONSTRUCTOR SubscriptionGUI()
   Create GUI window
   Initialize inputs, buttons, and output area
   Register action listeners
   Display window
END CONSTRUCTOR

METHOD actionPerformed(event)

   IF Add Personal Plan button clicked THEN
      Read input values
      Create PersonalPlan object
      Add to plans list
      Display confirmation message
   END IF

   IF Add Pro Plan button clicked THEN
      Read input values
      Create ProPlan object
      Add to plans list
      Display confirmation message
   END IF

   IF Enter Prompt button clicked THEN
      Read plan index
      Get plan from plans list
      IF plan is PersonalPlan THEN
         Call enterPrompt method
         Display result
      ELSE
         Display error message
      END IF
   END IF

   IF Add Team Member button clicked THEN
      Read plan index
```

```
        Get plan from plans list
        IF plan is ProPlan THEN
            Call addTeamMember method
            Display result
        ELSE
            Display error message
        END IF
    END IF

    IF Display All button clicked THEN
        FOR each plan in plans list
            Display index and plan details
        END FOR
    END IF

    IF Save button clicked THEN
        Write plan details to text file
        Display success message
    END IF

    IF Load button clicked THEN
        Read data from text file
        Display loaded data
    END IF

    IF Clear button clicked THEN
        Clear all input fields
        Display confirmation message
    END IF

END METHOD

METHOD main()
    Create SubscriptionGUI object
END METHOD

END CLASS
```

4. Method Description

In this section we will be discussing about the method in each class. Describing each method purpose and functionality implemented in each class. For each method, name, Return Type, Parameters are presented in the table below followed by detailed internal working explanation (Barnes & Kölling, 2021).

4.1 AIModel.java(Abstract parent class)

Method Name	Parameter Type	Return Type	Description	Internal Working
AIModel()	String, double, int, int	None	Constructor used to initialize common attributes of AI models	Assigns model name, price, parameter count, and context window to instance variables
getModelName()	None	String	Returns the name of the AI model	Simply returns the value stored in the modelName variable
getPrice()	None	double	Returns the price of the AI model	Returns the value of the price attribute
getParameterCount()	None	int	Returns the parameter count of the model	Returns the parameterCount variable
getContextWindow()	None	int	Returns the context window size	Returns the contextWindow value
display()	None	String	Displays details of the model	Declared as abstract; implemented by child classes

4.2 PersonalPlan.java

Method Name	Parameter Type	Return Type	Description	Internal Working
PersonalPlan()	String, double, int, int, int	None	Constructor for creating a personal plan	Calls parent constructor and sets available tokens
enterPrompt()	String, int	String	Allows user to submit a prompt using tokens	Counts input tokens, adds expected tokens, checks limits, deducts tokens if valid
display()	None	String	Displays personal plan details	Returns model information along with remaining available tokens

4.3 ProPlan.java

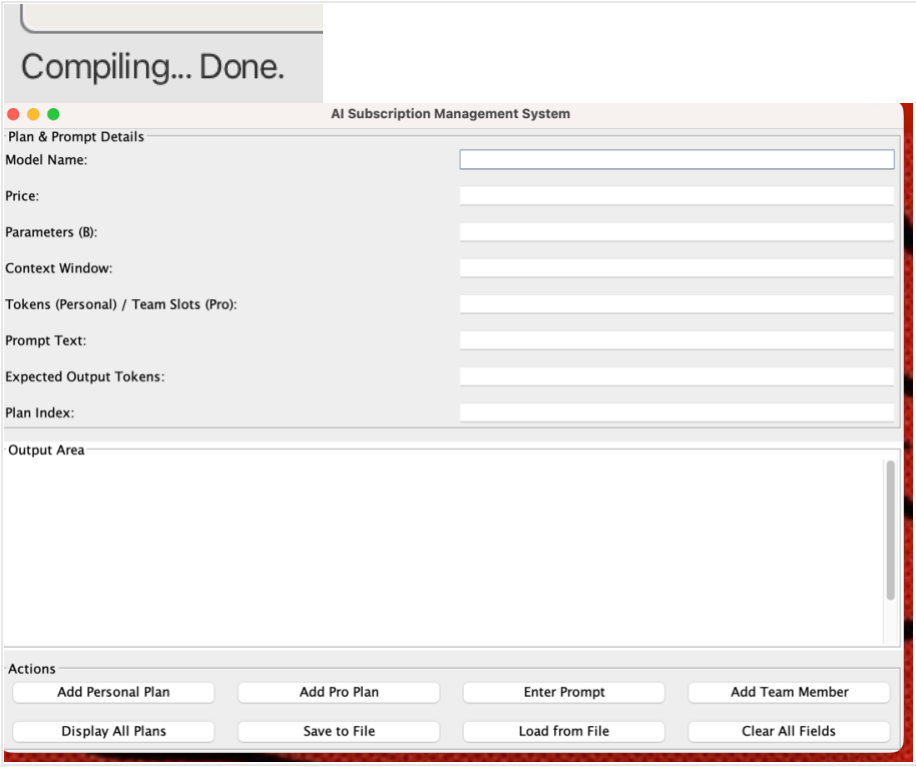
Method Name	Parameter Type	Return Type	Description	Internal Working
ProPlan()	String, double, int, int, int	None	Constructor for creating a pro plan	Calls parent constructor and assigns team slots
addTeamMember()	String	String	Adds a team member to the pro plan	Checks available slots and reduces slot count if possible
display()	None	String	Displays pro plan details	Returns model information with available team slots

4.4 SubscriptionGUI.java

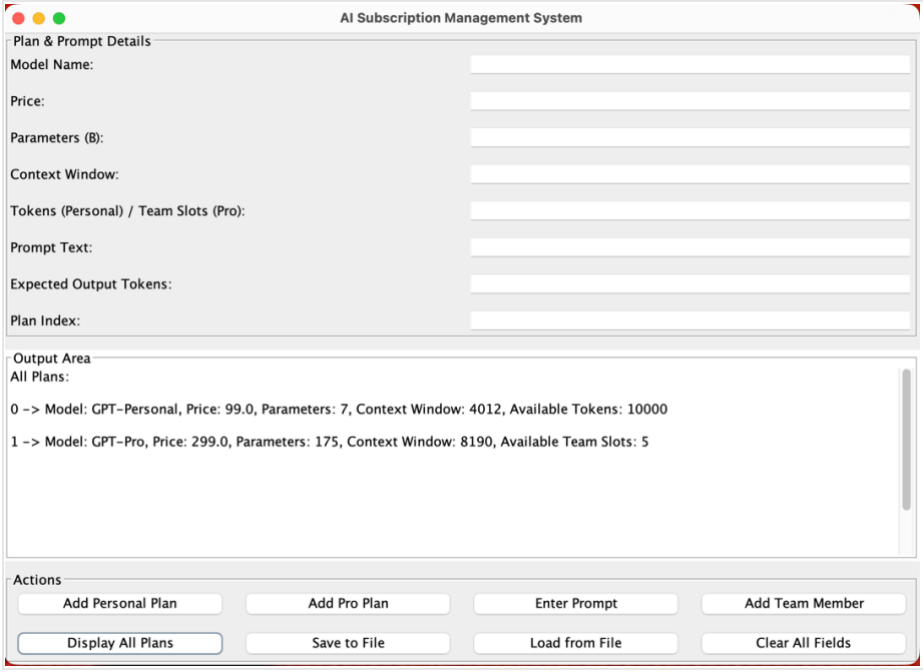
Method Name	Parameter Type	Return Type	Description	Internal Working
SubscriptionGUI	None	None	Constructs and initializes the graphical user interface	Creates input fields, buttons, output area, registers action listeners, and displays the window
actionPerformed	ActionEvent	void	Handles all button click events from the GUI	Identifies which button is clicked and performs corresponding actions such as adding plans, entering prompts, saving or loading data, and clearing input fields
main	String[]	void	Entry point of the program	Creates an instance of SubscriptionGUI to start the application

5. Testing

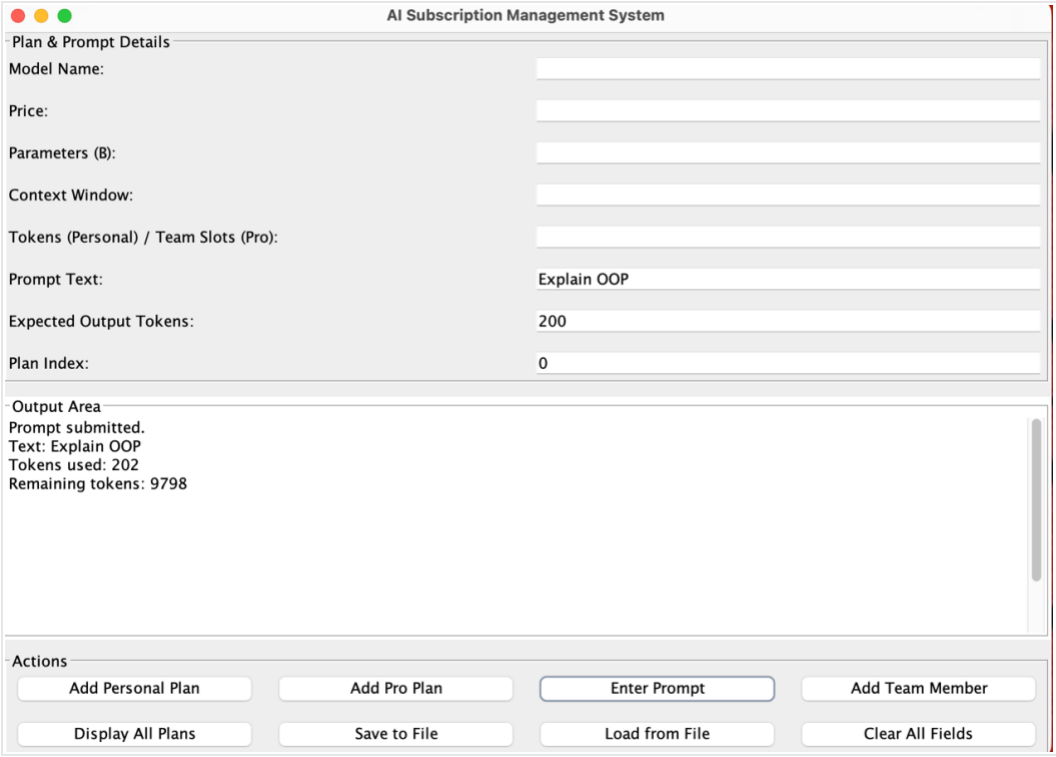
5.1 Test case 1: Compile and Run Program Using CLI

Field	Description
Objective	To verify that the program compiles and runs successfully
Steps Performed	The program was compiled and executed using BlueJ, which internally uses the Java command-line compiler
Expected Result	Program should compile without errors and the GUI window should launch
Actual Result	Program compiled successfully and GUI window opened
Evidence	

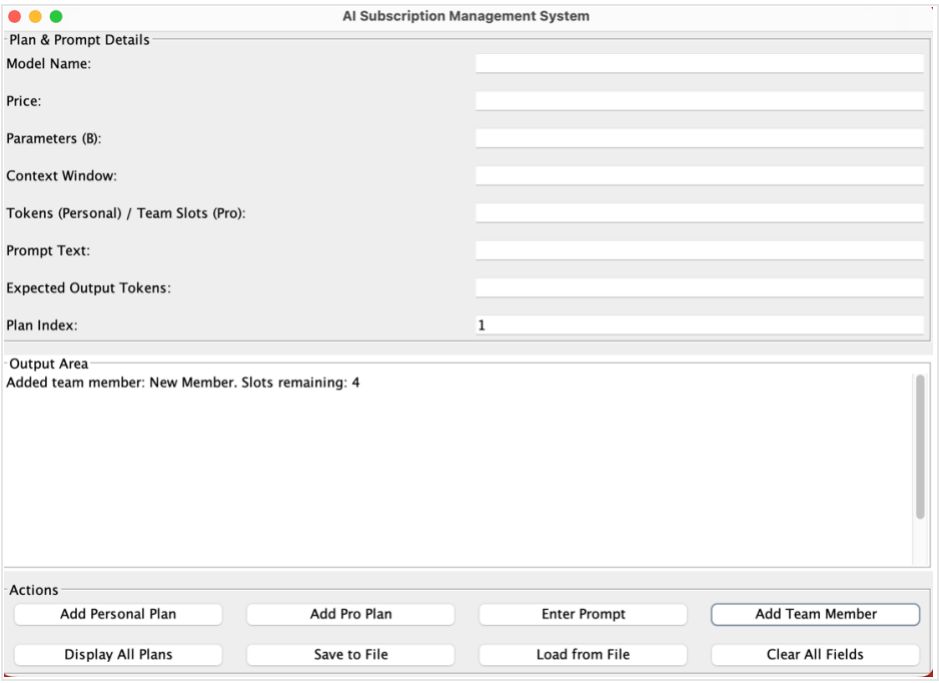
5.2 Test case 2: Add Personal and Pro Plan to ArrayList

Field	Description
Objective	To verify that Personal and Pro plans can be added to the ArrayList
Steps Performed	Entered valid details and clicked “Add Personal Plan” and “Add Pro Plan” buttons
Expected Result	Both plans should be added and displayed with correct index values
Actual Result	Both plans were successfully added and displayed correctly
Evidence	

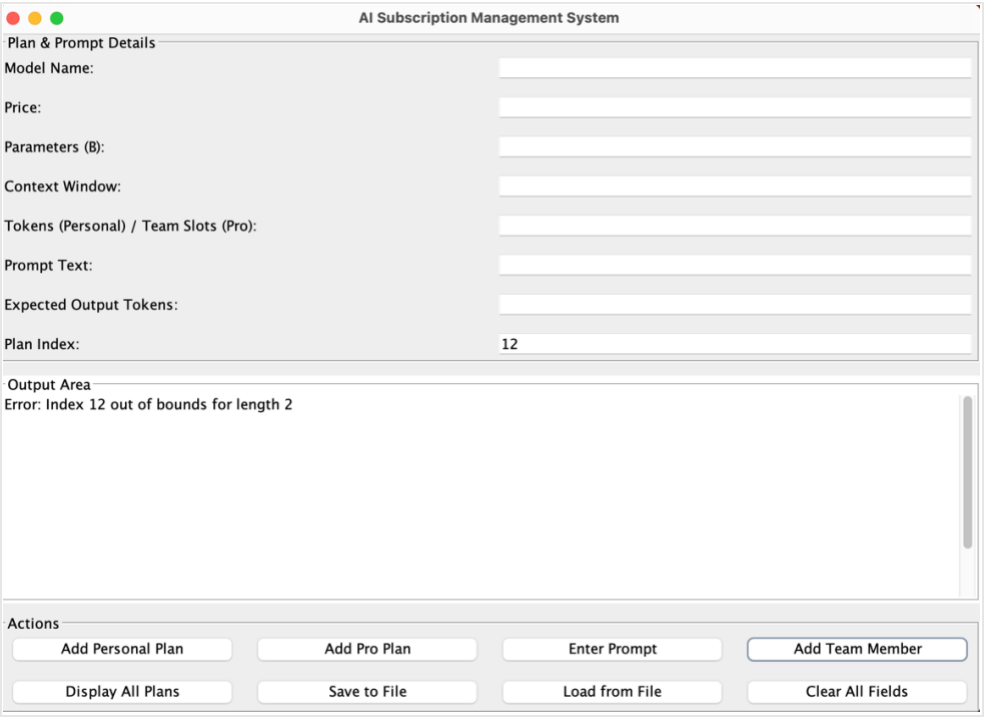
5.3 Test case 3: Make a Prompt

Field	Description
Objective	To verify that a prompt can be submitted using a Personal Plan
Steps Performed	Entered prompt text, expected output tokens, and valid personal plan index, then clicked “Enter Prompt”
Expected Result	Tokens should be calculated and deducted from the personal plan
Actual Result	Prompt processed successfully and remaining tokens displayed
Evidence	

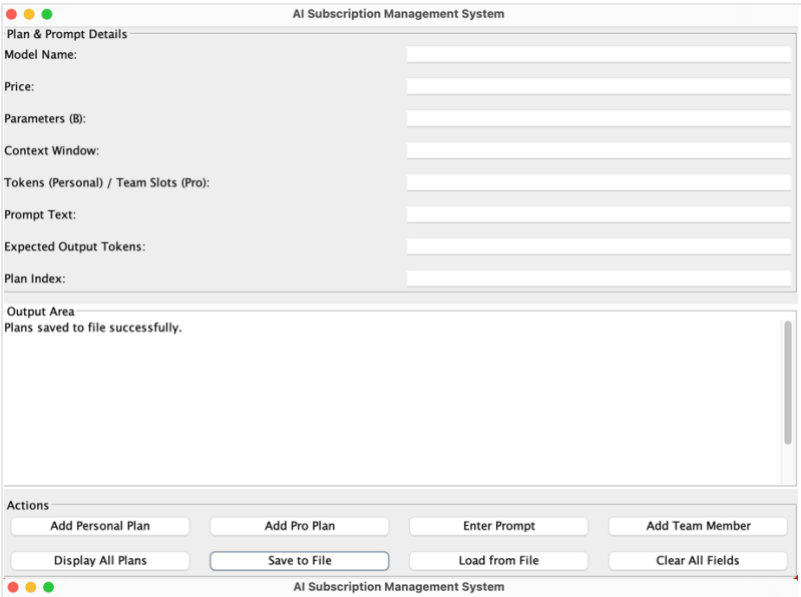
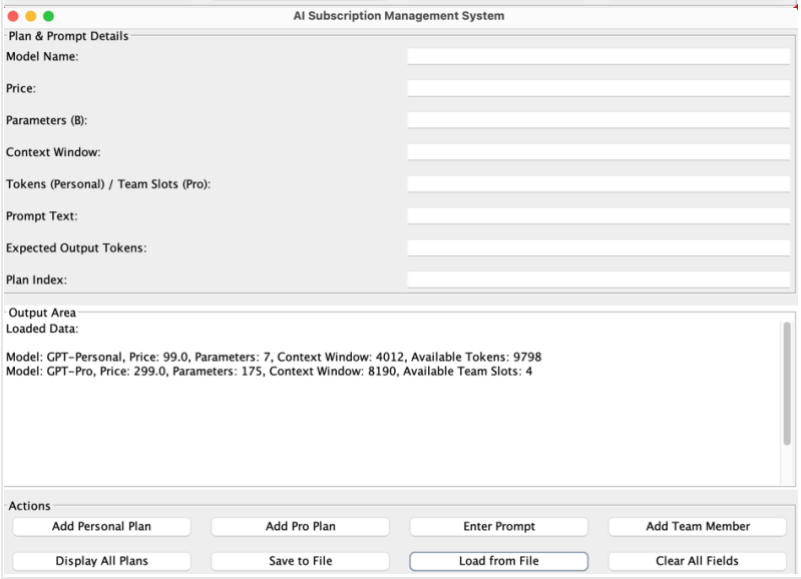
5.4 Test case 4: Add Team Member

Field	Description
Objective	To verify that a team member can be added to a Pro Plan
Steps Performed	Entered valid pro plan index and clicked “Add Team Member”
Expected Result	Team member should be added and available team slots reduced
Actual Result	Team member added successfully and team slots reduced
Evidence	

5.5 Test case 5: Invalid Index Value Entered

Field	Description
Objective	To verify how the system handles an invalid index value
Steps Performed	Entered an invalid index value and attempted an operation
Expected Result	System should display an error message without crashing
Actual Result	Error message displayed correctly
Evidence	

5.6 Test case 6: Save and Load File

Field	Description
Objective	To verify file saving and loading functionality
Steps Performed	Clicked “Save to File” followed by “Load from File”
Expected Result	Data should be saved and loaded successfully
Actual Result	Data was saved to file and loaded correctly in the output area
Evidence	 

6. Error Detection And Correction

Error detection and correction is an important part of software development and helps improve program reliability and robustness (Barnes & Kölling, 2021). During the implementation of this system, syntax, runtime, and logical errors were identified through compilation messages, execution results, and testing. These errors were resolved by correcting code structure, handling invalid user input using exception handling, and fixing conditional logic, which improved the overall correctness and robustness of the application (Deitel & Deitel, 2018).

6.1 Syntax error

Field	Description
Objective	To demonstrate detection and correction of a syntax error in Java

Error (Screenshot)	A compilation error occurred due to a missing semicolon (;) at the end of a statement in the AIModel class.
	`public AIModel(String modelName, double price, int para`
	`this.modelName = modelName;`
	`this.price = price;`
	`this.parameterCount = parameterCount;`
	`this.contextWindow = contextWindow`
	`}`
	`';' expected`
	`// Getter methods`

Reason	Java requires every statement to end with a semicolon. Omitting it causes a compilation error
Correction	The missing semicolon was added at the end of the statement, allowing the program to compile successfully

Evidence	`this.parameterCount = parameterCount;`
	`this.contextWindow = contextWindow;`
	Class compiled - no syntax errors

6.2 Runtime error

Field	Description
Objective	To demonstrate detection and correction of a runtime error in Java
Error (Screenshot)	Runtime error occurs when non-numeric input is entered in a numeric field, causing NumberFormatException 
Reason	Double.parseDouble() was unable to convert text input into a number
Correction	User was instructed to enter valid numeric values, and exception handling prevents program crash
Evidence	

6.3 Logical error

Field	Description
Objective	To demonstrate a logical error in conditional checking

Error (Screenshot)	Program incorrectly allowed a Pro Plan to use prompt functionality
	`if (plan instanceof ProPlan) {  // WRONG LOGIC`
	`  ProPlan pr = (ProPlan) plan;`
	`  outputArea.setText("Prompt accepted.");`
	`} else {`
	`  outputArea.setText("Prompt is allowed only for Personal Plans.");`
	`}`

Reason	Wrong conditional logic was used to check the plan type
Correction	Corrected the condition to allow only PersonalPlan to submit prompts

Evidence	`if (plan instanceof PersonalPlan) {`
	`  PersonalPlan pp = (PersonalPlan) plan;`
	`  outputArea.setText(`
	`  pp.enterPrompt(`
	`  promptField.getText(),`
	`  Integer.parseInt(expectedTokensField.getText()))`
	`  );`
	`} else {`
	`  outputArea.setText("Prompt is allowed only for Personal Plans.");`
	`}`

7. Conclusion and Reflection

In summary, this paper thoroughly illustrated the usage of object-oriented programming (OOP) concepts in the creation of an AI Subscription Management System with Java. The system was structured around fundamental OOP principles like abstraction, inheritance, and polymorphism. Besides, the enhanced version of the system featured a user-friendly interface as well as data storage and retrieval capabilities through file handling operations.

The functionalities required for the coursework were successfully implemented and tested. This program accomplishes the goals that were mentioned in the specification of the coursework. Through this task, I understood more about planning class hierarchies, utilizing abstract classes, and controlling objects through an ArrayList. Constructing the graphical user interface (GUI) with the aid of Java Swing enabled me to raise my level of expertise in managing user input, event handling, and effectively communicating with the help of error messages.

Besides, I understood the significance of verifying user input and appropriately dealing with runtime and logical errors for smooth program operation. Furthermore, preparing test cases and finding errors sharpened my abilities to resolve problems and made me comprehend programming errors that are very common such as syntax errors, runtime exceptions, and logical errors. Generally, this project increased my assurance in doing Java programming and enriched my grasp of how theoretical notions are implemented in actual software creating.

8. References

Barnes, D. J. & Kölling, M., 2021. *Objects First with Java.* 6th ed. Harlow: Pearson Education.
Oracle, 2024. *Java Platform, Standard Edition Documentation.* [Online]
Available at: https://docs.oracle.com/en/java/
[Accessed 15 April 2026].
Deitel, P. & Deitel, H., 2018. *Java How to Program.* 11th ed. Harlow: Pearson Education.
Object Management Group, 2023. *UML Class Diagrams.* [Online]
Available at: https://www.omg.org/spec/UML
[Accessed 14 April 2026].

9. Appendix

This appendix contains the complete source code listings for the AI Subscription Management System developed for this assignment. The code includes the abstract parent class, child classes, and the graphical user interface class.

9.1 Appendix 1: AIModel.java

```
package assignment;


// Parent class for all AI models
public abstract class AIModel {

    // Common variables for all plans
    private String modelName;
    private double price;
    private int parameterCount;
    private int contextWindow;

    // Constructor
    public AIModel(String modelName, double price, int parameterCount, int contextWindow) {
        this.modelName = modelName;
        this.price = price;
        this.parameterCount = parameterCount;
        this.contextWindow = contextWindow;
    }

    // Getter methods
    public String getModelName() {
        return modelName;
    }

    public double getPrice() {
        return price;
    }

    public int getParameterCount() {
        return parameterCount;
    }
```

```
    public int getContextWindow() {
        return contextWindow;
    }

    // Display method to be implemented by child classes
    public abstract String display();
}
```

9.2 Appendix 2: PersonalPlan.java

```
package assignment;

// Personal plan class
public class PersonalPlan extends AIModel {

    private int availableTokens;

    // Constructor
    public PersonalPlan(String modelName, double price, int parameterCount,
                        int contextWindow, int availableTokens) {
        super(modelName, price, parameterCount, contextWindow);
        this.availableTokens = availableTokens;
    }

    // Method to enter a prompt
    public String enterPrompt(String promptText, int expectedTokens) {

        int inputTokens = promptText.split(" ").length;
        int totalTokens = inputTokens + expectedTokens;

        if (totalTokens > getContextWindow()) {
            return "Error: Token limit exceeds context window.";
        }

        if (availableTokens >= totalTokens) {
            availableTokens -= totalTokens;
            return "Prompt submitted.\nText: " + promptText +
                   "\nTokens used: " + totalTokens +
                   "\nRemaining tokens: " + availableTokens;
        } else {
            return "Not enough available tokens.";
        }
    }
```

```
    // Display plan information
    @Override
    public String display() {
        return "Model: " + getModelName() +
                ", Price: " + getPrice() +
                ", Parameters: " + getParameterCount() +
                ", Context Window: " + getContextWindow() +
                ", Available Tokens: " + availableTokens;
    }
}
```

9.3 Appendix 3: ProPlan.java

```
package assignment;

// Pro plan class
public class ProPlan extends AIModel {

    private int availableTeamSlots;

    // Constructor
    public ProPlan(String modelName, double price, int parameterCount,
                   int contextWindow, int availableTeamSlots) {
        super(modelName, price, parameterCount, contextWindow);
        this.availableTeamSlots = availableTeamSlots;
    }

    // Method to add team member
    public String addTeamMember(String memberName) {
        if (availableTeamSlots > 0) {
            availableTeamSlots--;
            return "Added team member: " + memberName +
                    ". Slots remaining: " + availableTeamSlots;
        } else {
            return "No available team slots.";
        }
    }

    // Display plan information
    @Override
    public String display() {
        return "Model: " + getModelName() +
                ", Price: " + getPrice() +
                ", Parameters: " + getParameterCount() +
```

```
            ", Context Window: " + getContextWindow() +
            ", Available Team Slots: " + availableTeamSlots;
    }
}
```

9.4 Appendix 4: SubscriptionGUI.java

```
package assignment;



import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.io.*;

public class SubscriptionGUI extends JFrame implements ActionListener {

    // Store all plans
    private ArrayList<AIModel> plans = new ArrayList<>();

    // Input fields
    private JTextField modelNameField, priceField, paramField, contextField;
    private JTextField tokensOrSlotsField, promptField, expectedTokensField, indexField;

    // Buttons
    private JButton addPersonalBtn, addProBtn, promptBtn, teamBtn;
    private JButton displayBtn, saveBtn, loadBtn, clearBtn;

    // Output area
    private JTextArea outputArea;

    public SubscriptionGUI() {

        setTitle("AI Subscription Management System");
        setSize(900, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        // INPUT PANEL
        JPanel inputPanel = new JPanel(new GridLayout(8, 2, 10, 10));
        inputPanel.setBorder(BorderFactory.createTitledBorder("Plan & Prompt Details"));

        modelNameField = new JTextField();
```

```
priceField = new JTextField();
paramField = new JTextField();
contextField = new JTextField();
tokensOrSlotsField = new JTextField();
promptField = new JTextField();
expectedTokensField = new JTextField();
indexField = new JTextField();

inputPanel.add(new JLabel("Model Name:"));
inputPanel.add(modelNameField);

inputPanel.add(new JLabel("Price:"));
inputPanel.add(priceField);

inputPanel.add(new JLabel("Parameters (B):"));
inputPanel.add(paramField);

inputPanel.add(new JLabel("Context Window:"));
inputPanel.add(contextField);

inputPanel.add(new JLabel("Tokens (Personal) / Team Slots (Pro):"));
inputPanel.add(tokensOrSlotsField);

inputPanel.add(new JLabel("Prompt Text:"));
inputPanel.add(promptField);

inputPanel.add(new JLabel("Expected Output Tokens:"));
inputPanel.add(expectedTokensField);

inputPanel.add(new JLabel("Plan Index:"));
inputPanel.add(indexField);

add(inputPanel, BorderLayout.NORTH);

//  OUTPUT AREA
outputArea = new JTextArea(15, 60);
outputArea.setEditable(false);
outputArea.setLineWrap(true);
outputArea.setWrapStyleWord(true);

JScrollPane scrollPane = new JScrollPane(outputArea);
scrollPane.setBorder(BorderFactory.createTitledBorder("Output Area"));

add(scrollPane, BorderLayout.CENTER);

//  BUTTON PANEL
```

```
    JPanel buttonPanel = new JPanel(new GridLayout(2, 4, 10, 10));
    buttonPanel.setBorder(BorderFactory.createTitledBorder("Actions"));

    addPersonalBtn = new JButton("Add Personal Plan");
    addProBtn = new JButton("Add Pro Plan");
    promptBtn = new JButton("Enter Prompt");
    teamBtn = new JButton("Add Team Member");
    displayBtn = new JButton("Display All Plans");
    saveBtn = new JButton("Save to File");
    loadBtn = new JButton("Load from File");
    clearBtn = new JButton("Clear All Fields");

    buttonPanel.add(addPersonalBtn);
    buttonPanel.add(addProBtn);
    buttonPanel.add(promptBtn);
    buttonPanel.add(teamBtn);
    buttonPanel.add(displayBtn);
    buttonPanel.add(saveBtn);
    buttonPanel.add(loadBtn);
    buttonPanel.add(clearBtn);

    add(buttonPanel, BorderLayout.SOUTH);

    // Register actions
    addPersonalBtn.addActionListener(this);
    addProBtn.addActionListener(this);
    promptBtn.addActionListener(this);
    teamBtn.addActionListener(this);
    displayBtn.addActionListener(this);
    saveBtn.addActionListener(this);
    loadBtn.addActionListener(this);
    clearBtn.addActionListener(this);

    setVisible(true);
}

//  BUTTON LOGIC
@Override
public void actionPerformed(ActionEvent e) {

    try {

        if (e.getSource() == addPersonalBtn) {

            PersonalPlan p = new PersonalPlan(
                    modelNameField.getText(),
```

```
            Double.parseDouble(priceField.getText()),
            Integer.parseInt(paramField.getText()),
            Integer.parseInt(contextField.getText()),
            Integer.parseInt(tokensOrSlotsField.getText())
        );

        plans.add(p);
        outputArea.setText("Personal Plan added successfully.");
    }

    else if (e.getSource() == addProBtn) {

        ProPlan p = new ProPlan(
            modelNameField.getText(),
            Double.parseDouble(priceField.getText()),
            Integer.parseInt(paramField.getText()),
            Integer.parseInt(contextField.getText()),
            Integer.parseInt(tokensOrSlotsField.getText())
        );

        plans.add(p);
        outputArea.setText("Pro Plan added successfully.");
    }

    else if (e.getSource() == displayBtn) {

        outputArea.setText("All Plans:\n\n");
        for (int i = 0; i < plans.size(); i++) {
            outputArea.append(i + " -> " + plans.get(i).display() + "\n\n");
        }
    }

    else if (e.getSource() == promptBtn) {

        int index = Integer.parseInt(indexField.getText());
        AIModel plan = plans.get(index);

        if (plan instanceof PersonalPlan) {
            PersonalPlan pp = (PersonalPlan) plan;
            outputArea.setText(
                pp.enterPrompt(
                    promptField.getText(),
                    Integer.parseInt(expectedTokensField.getText()))
            );
        } else {
            outputArea.setText("Prompt is allowed only for Personal Plans.");
```

```
    }
}

else if (e.getSource() == teamBtn) {

    int index = Integer.parseInt(indexField.getText());
    AIModel plan = plans.get(index);

    if (plan instanceof ProPlan) {
        ProPlan pr = (ProPlan) plan;
        outputArea.setText(pr.addTeamMember("New Member"));
    } else {
        outputArea.setText("Team management is available only for Pro Plans.");
    }
}

else if (e.getSource() == saveBtn) {

    PrintWriter pw = new PrintWriter("Assignment/plans.txt");
    for (AIModel m : plans) {
        pw.println(m.display());
    }
    pw.close();

    outputArea.setText("Plans saved to file successfully.");
}

else if (e.getSource() == loadBtn) {

    BufferedReader br = new BufferedReader(
            new FileReader("Assignment/plans.txt"));

    outputArea.setText("Loaded Data:\n\n");
    String line;
    while ((line = br.readLine()) != null) {
        outputArea.append(line + "\n");
    }
    br.close();
}

else if (e.getSource() == clearBtn) {

    modelNameField.setText("");
    priceField.setText("");
    paramField.setText("");
    contextField.setText("");
```

```
            tokensOrSlotsField.setText("");
            promptField.setText("");
            expectedTokensField.setText("");
            indexField.setText("");

            outputArea.setText("All input fields cleared.");
        }

      }
      catch (Exception ex) {
         outputArea.setText("Error: " + ex.getMessage());
      }
   }

   public static void main(String[] args) {
      new SubscriptionGUI();
   }
}
```